The Journal of Open Source Software

HoverFast: an accurate, high-throughput, clinically deployable nuclear segmentation tool for brightfield digital pathology images

Petros Liakopoulos[1*¶], **Julien Massonnet**[2*], **Jonatan Bonjour**[1], **Medya Tekes Mizrakli**[3], **Simon Graham**[4], **Michel A. Cuendet**[1], **Amanda H. Seipel**[2], **Olivier Michielin**[1], **Doron Merkler**[2], **and Andrew Janowczyk**[1,2,5]

1 Service of Precision Oncology, Department of Oncology, University of Geneva and Geneva University Hospitals, Geneva, Switzerland **2** Division of Clinical Pathology, , Departments of Pathology and Immunology & Diagnostics, University of Geneva and Geneva University Hospitals, Geneva, Switzerland **3** Section of Communication Systems, School of Computer and Communication Sciences, École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland. **4** Histofy Ltd, Birmingham, United Kingdom **5** Emory University, Atlanta, Georgia, United States of America. ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 24 May 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

In computational digital pathology, accurate nuclear segmentation of Hematoxylin and Eosin (H&E) stained whole slide images (WSIs) is a critical step for many analyses and tissue characterizations. One popular deep learning-based nuclear segmentation approach, HoverNet (Graham et al., 2019), offers remarkably accurate results but lacks the high-throughput performance needed for clinical deployment in resource-constrained settings. Our approach, HoverFast, aims to provide fast and accurate nuclear segmentation in H&E images using knowledge distillation from HoverNet. By redesigning the tool with software engineering best practices, HoverFast introduces advanced parallel processing capabilities, efficient data handling, and optimized postprocessing. These improvements facilitate scalable high-throughput performance, making HoverFast more suitable for real-time analysis and application in resource-limited environments. Using a consumer grade Nvidia A5000 GPU, HoverFast showed a 21x speed improvement as compared to HoverNet; reducing mean analysis time for 40x WSIs from ~2 hours to 6 minutes while retaining a concordant mean Dice score of 0.91 against the original HoverNet output. Peak memory usage was also reduced 71% from 44.4GB, to 12.8GB, without requiring SSD-based caching. To ease adoption in research and clinical contexts, HoverFast aligns with best-practices in terms of (a) installation, and (b) containerization, while (c) providing outputs compatible with existing popular open-source image viewing tools such as QuPath (Bankhead et al., 2017). HoverFast has been made open-source and is available at andrewjanowczyk.com/open-source-tools/hoverfast.

Statement of need

The increasing popularity of digitized pathology images in both research and clinical practice has spurred the widespread adoption of deep learning (DL) approaches for automating various tasks, with nuclear segmentation standing out as a crucial step in many analyses. This segmentation process involves delineating the contours of cell nuclei within a 2D whole slide image (WSI). Nuclei, rather than complete cells, are targeted due to strong contrast afforded by routinely employed hematoxylin staining. Hematoxylin's selective affinity for nucleic acids results in the distinct visualization of nuclei in purple, facilitating their clear identification

amidst less prominently stained cytoplasm and other cellular constituents. Given the small size of nuclei, their segmentation typically takes place at 40x magnification (~0.25 microns per pixel (mpp)); the highest magnification supported by most current digital slide scanners. Working at this scale can be time-intensive for algorithms, especially on consumer grade GPUs, as WSIs are especially large, reaching up to 120,000x120,000 pixels. While several existing tools like StarDist (Schmidt et al., 2018) (Weigert et al., 2020) and CellPose (Stringer et al., 2021) have been developed to tackle the challenge of nuclear segmentation, HoverNet(Graham et al., 2019) has emerged as one of the leading solutions in terms of segmentation accuracy, particularly for its application to H&E-stained tissue.

Despite its accurate results, HoverNet remains resource-intensive and time-consuming due to its high model parameter count and lengthy post-processing steps. HoverNet additionally requires significant SSD storage for caching during runtime, often reaching over 120GB per WSI. These properties make it challenging to deploy in more resource limited settings such as consumer grade workstations or in clinical environments requiring high-throughput processing. Therefore, there is an emerging need for a fast, accurate, and computationally efficient tool that can make large-scale nuclear segmentation more accessible for both research and clinical applications.

Motivated by the need for accurate yet efficient nuclear segmentation, we introduce HoverFast. This tool replicates the output of the established HoverNet model while achieving superior computational efficiency. HoverFast achieves this through knowledge distillation, a technique where a smaller "student" model (HoverFast) learns to capture the knowledge from a larger "teacher" model (HoverNet). The goal is to enable the student model to achieve comparable performance to that of the teacher model, while requiring significantly less computational resources for inference (Hinton et al., 2015), (Hu et al., 2022).

To facilitate the knowledge distillation process, HoverFast presents a training pipeline, using HoverNet output as ground truth (see **Figure 1**), that enables the resulting model to have 30 times fewer parameters. As implemented, HoverFast provides:

- A containerized docker script to generate HoverNet ground truth on user-provided data
- A training pipeline for a custom HoverFast model
- Alternatively, a pre-trained cross-organ model for inference
- An inference pipeline for tiles and WSIs with tissue masks to delineate area of computation
- Compressed JSON output file directly compatible with QuPath
- A speedup of 21x over HoverNet, on consumer-grade compute infrastructure

DRAFT

The Journal of Open Source Software

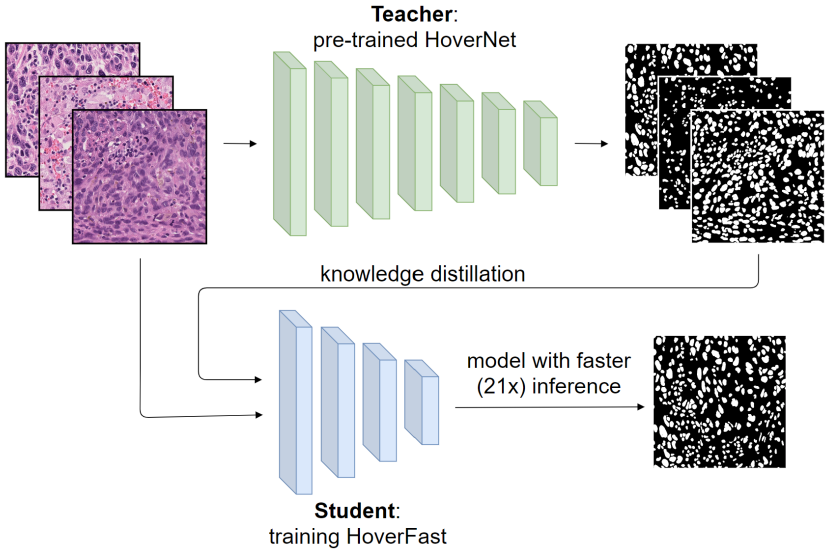

Figure 1: Overview of training pipeline for HoverFast. H&E tiles are segmented using HoverNet. The nuclei masks, as well as the original H&E tiles, are passed to the smaller HoverFast architecture for training. The resulting HoverFast model provides a highly optimized inference and post-processing framework that can then be used for nuclear segmentation on WSIs with a 21x speed improvement over HoverNet.

Implementation

Inference

HoverFast has a command-line interface (CLI) written in Python 3.11 and utilizes the PyTorch framework (Paszke et al., 2017). We replicated the structure of the HoverNet model as described by Graham et al (Graham et al., 2019) without the nuclear classification branch. For the backbone, we used a modified 940k parameter Multi-scale UNet (Su et al., 2021) in place of HoverNet's 33.6 million parameter ResNet50, yielding a reduction in model parameter count by a factor of 30 (see **Appendix 1**).

HoverFast's post-processing pipeline was heavily optimized using scikit-learn's (Pedregosa et al., 2011) regionprops and watershed functions to effectively identify and split merged cells. To improve throughput after batch model inference, regions are processed in parallel using a "multi-worker, single writer" approach. This involves each worker independently (a) post-processing its assigned region, and then (b) generating nuclei polygon coordinates using OpenCV (Bradski, 2000), before (c) sending to the single writing process for saving as a QuPath (Bankhead et al., 2017) compatible gzip-compressed JSON file. A Docker and Singularity container of HoverFast are provided.

Training

To help users train their own models, we provide a Docker container with HoverNet installed, and a script that (a) accepts a directory of WSIs (or tiles), (b) randomly extracts a user-specified number of tiles, (c) employs HoverNet on these tiles to generate labeled masks of nuclei, and finally (d) saves the original images and associated masks into two PyTables files, one for training and one for validation. HoverFast can then accept these PyTables files as arguments in its training script to yield a use-case specific model. We employ knowledge distillation during HoverFast training using the same loss function as HoverNet. The teacher model, HoverNet, guides the student model, HoverFast, by providing the ground truth binary mask as target.

DRAFT

Additionally, HoverFast learns to reproduce the horizontal and vertical distance maps allowing HoverFast to inherit HoverNet's post-processing abilities for separating touching nuclei.

Experiments

Experiment 1: Comparison of HoverNet and HoverFast on a cross organ dataset

Employing n=97 WSIs of diverse tissue types, from The Cancer Genome Atlas (TCGA) (Weinstein et al., 2013), 15 randomly selected tiles of 1,024x1,024 pixels from each WSI were extracted. A HoverFast model was then trained as described in Training, for 100 epochs with a batch size of 16. For validation, 74 tiles of 1,024x1,024 pixels from 14 slides of diverse tissue types were used. Inference using both HoverNet and HoverFast was performed on the validation tiles, and the binary masks of predicted cell nuclei were overlapped to obtain a Dice score between the two tools. The resulting Dice score of 0.91 appears concordant with the qualitative results (see **Figure 2**); these show very similar segmentation results, with HoverFast able to segment slightly more faint nuclei than HoverNet.

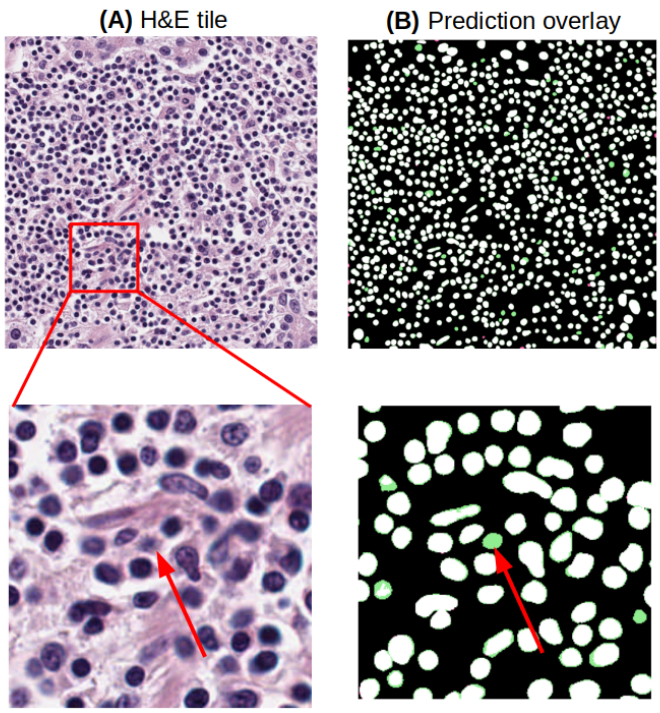

Figure 2: (A) 1,024x1,024 tile of H&E tissue. (B) Overlay of binary masks. Pixels in white are predicted by both models, with those in green/pink predicted only by HoverFast/HoverNet, respectively. For this patch, the Dice score is 0.91. Here we can generally see that while HoverFast detects more nuclei, those predictions remain reasonable and clearly visible on H&E, suggesting a potentially higher quality result.

Experiment 2: Comparison of HoverNet, cross-tissue HoverFast, and site-specific HoverFast

From n=54 melanoma samples, (a) for training: 20 1,024x1,024 tiles were randomly selected per slide from within available tumor masks, and (b) for validation: 50 tiles of 1,024x1,024

from 8 slides were selected. For evaluation, 3 models were compared: (i) HoverNet, as a baseline, (ii) the HoverFast model trained in Experiment 1, and (iii) a melanoma specific model trained following the procedure in Training. The HoverFast models had Dice scores of 0.88 and 0.91 respectively against HoverNet, with qualitative results indicating a high degree of similarity. There were slight changes on nuclei edges and faint nuclei (see **Figure 3**), with a systematic superiority for the tissue-specific output. Taken together, the increased accuracy in the melanoma specific model demonstrates that investing in training a dataset-specific model appears to provide added value.

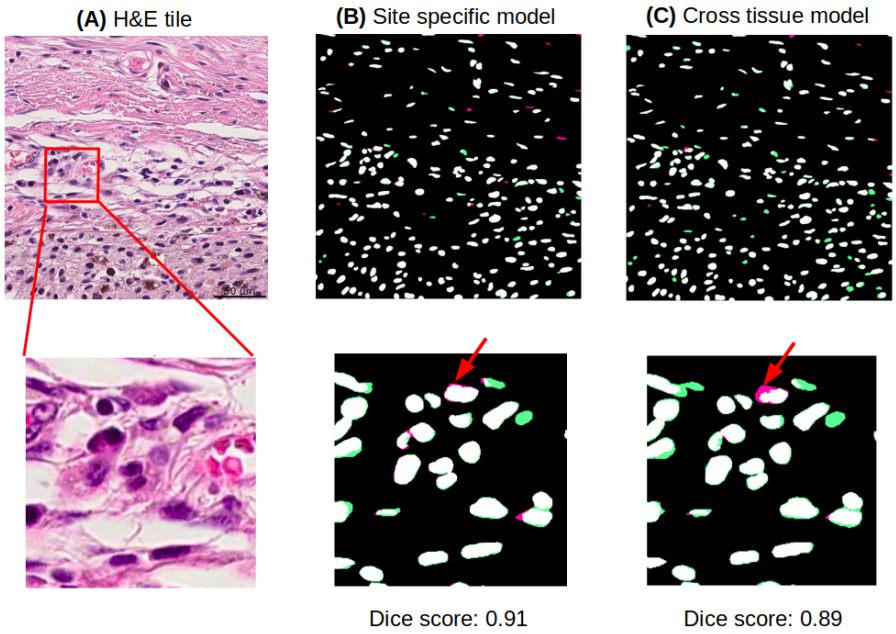

Figure 3: (A) 1,024x1,024 tile of H&E tissue (top) with a higher magnification region of interest (bottom). (B) Overlay of binary masks for the single tissue model. Pixels in white are predicted by both models, with those in green/pink predicted only by HoverFast/HoverNet, respectively. (C) Overlay of binary mask for the cross-tissue model. The tissue specific model shows a closer resemblance to HoverNet in terms of cell outlines.

Experiment 3: Benchmarks comparing processing time and memory footprint

To compare computational speed, n=4 slides from TCGA with corresponding tissue masks generated with HistoQC (Janowczyk et al., 2019) were analyzed on a machine with a 16 core Intel(R) Core(TM) i9-12900K CPU, a Nvidia A5000 GPU with 24GB of VRAM, and 128Gb of DDR5 RAM. For both HoverNet and HoverFast, the GPU batch size was set to maximize GPU memory usage. For HoverNet, a batch size of 90 was used, with 20 CPU threads for pre- and post-processing. Similarly, for HoverFast, a batch size of 11 and 20 CPU threads were used. A mean speed improvement of 20.8x times (see **Table 1**) was demonstrated. The maximum RAM consumption was reduced by 71% with 44.4 GB for HoverNet versus 12.8 GB for HoverFast. Additionally, HoverNet required a peak of 118 GB of SSD space for its cache during run-time, while HoverFast did not appear to require any.

Table 1: Detailed table of computation time per slide for each tool with associated speedup

Slide ID	*HoverNet*	*HoverFast*	*Speedup*
Slide 1	58mins 5s	3mins 1s	19.2x
Slide 2	1hr 11mins 38s	3mins 33s	20.2x
Slide 3	2hrs 55mins 24s	8mins 4s	21.7x
Slide 4	3hrs 4mins 25s	8mins 50s	20.9x
Total	**8hrs 9mins 32s**	**23mins 28s**	**20.8x**

Discussion and Conclusions:

HoverFast represents a practical solution to the challenge of nuclear segmentation in WSIs, emphasizing speed, resource efficiency and local trainability. It distinguishes itself by providing a significant speedup in processing time with a 21x improvement over HoverNet on consumer grade hardware in addition to a more than 3x reduction in RAM footprint while also eliminating hard-drive based caching. This efficiency is crucial for users with limited resources, enabling faster analysis while retaining segmentation results highly comparable to those of HoverNet. While a pre-trained cross-tissue model is provided with the software, if higher accuracy and greater similarity to HoverNet is required, a cohort specific model should be trained. Additionally, although HoverFast does have a built-in feature for tissue detection, we highly recommend the use of quality control tools, such as HistoQC to obtain more robust tissue masks, thus avoiding computation on artefactual regions and further reducing computation time. HoverFast is easy to install and provides simple drag and drop output compatibility with QuPath. It is publicly available for use and modification at andrewjanowczyk.com/open-source-tools/hoverfast.

DRAFT

Appendix 1

(A) HoverFast

```
==========================================================================
Layer (type:depth-idx)                   Param #
==========================================================================
HoverFast                                --
├─ModuleList: 1-1                        --
│    └─MSUNetConvBlock: 2-1              --
│    │    └─UNetConvBlock: 3-1           2,832
│    │    └─UNetConvBlock: 3-2           14,992
│    │    └─Conv2d: 3-3                  528
│    └─MSUNetConvBlock: 2-2              --
│    │    └─UNetConvBlock: 3-4           14,016
│    │    └─UNetConvBlock: 3-5           75,456
│    │    └─Conv2d: 3-6                  2,080
│    └─MSUNetConvBlock: 2-3              --
│    │    └─UNetConvBlock: 3-7           55,680
│    │    └─UNetConvBlock: 3-8           301,440
│    │    └─Conv2d: 3-9                  8,256
├─ModuleList: 1-2                        --
│    └─UNetUpBlock: 2-4                  --
│    │    └─ConvTranspose2d: 3-10        8,224
│    │    └─MSUNetConvBlock: 3-11        180,640
│    └─UNetUpBlock: 2-5                  --
│    │    └─ConvTranspose2d: 3-12        2,064
│    │    └─MSUNetConvBlock: 3-13        45,264
├─Conv2d: 1-3                            34
├─ModuleList: 1-4                        --
│    └─UNetUpBlock: 2-6                  --
│    │    └─ConvTranspose2d: 3-14        8,224
│    │    └─MSUNetConvBlock: 3-15        180,640
│    └─UNetUpBlock: 2-7                  --
│    │    └─ConvTranspose2d: 3-16        2,064
│    │    └─MSUNetConvBlock: 3-17        45,264
├─Conv2d: 1-5                            34
==========================================================================
Total params: 947,732
Trainable params: 947,732
Non-trainable params: 0
==========================================================================
```

(B) HoverNet

```
==========================================================================
Layer (type:depth-idx)                   Param #
==========================================================================
HoVerNet                                 --
├─Sequential: 1-1                        --
│    └─TFSamepaddingLayer: 2-1           --
│    └─Conv2d: 2-2                       9,408
│    └─BatchNorm2d: 2-3                  128
│    └─ReLU: 2-4                         --
├─ResidualBlock: 1-2                     --
│    └─ModuleList: 2-5                   --
│    │    └─Sequential: 3-1              57,600
│    │    └─Sequential: 3-2              70,400
│    │    └─Sequential: 3-3              70,400
│    └─Conv2d: 2-6                       16,384
│    └─Sequential: 2-7                   --
│    │    └─BatchNorm2d: 3-4             512
│    │    └─ReLU: 3-5                    --
├─ResidualBlock: 1-3                     --
│    └─ModuleList: 2-8                   --
│    │    └─Sequential: 3-6              246,272
│    │    └─Sequential: 3-7              280,064
│    │    └─Sequential: 3-8              280,064
│    │    └─Sequential: 3-9              280,064
│    └─Conv2d: 2-9                       131,072
│    └─Sequential: 2-10                  --
│    │    └─BatchNorm2d: 3-10            1,024
│    │    └─ReLU: 3-11                   --
├─ResidualBlock: 1-4                     --
│    └─ModuleList: 2-11                  --
│    │    └─Sequential: 3-12             984,064
│    │    └─Sequential: 3-13             1,117,184
│    │    └─Sequential: 3-14             1,117,184
│    │    └─Sequential: 3-15             1,117,184
│    │    └─Sequential: 3-16             1,117,184
│    │    └─Sequential: 3-17             1,117,184
│    └─Conv2d: 2-12                      524,288
│    └─Sequential: 2-13                  --
│    │    └─BatchNorm2d: 3-18            2,048
│    │    └─ReLU: 3-19                   --
├─ResidualBlock: 1-5                     --
│    └─ModuleList: 2-14                  --
│    │    └─Sequential: 3-20             3,934,208
│    │    └─Sequential: 3-21             4,462,592
│    │    └─Sequential: 3-22             4,462,592
│    └─Conv2d: 2-15                      2,097,152
│    └─Sequential: 2-16                  --
│    │    └─BatchNorm2d: 3-23            4,096
│    │    └─ReLU: 3-24                   --
├─Conv2d: 1-6                            2,097,152
├─ModuleDict: 1-7                        --
│    └─Sequential: 2-17                  --
│    │    └─Sequential: 3-25             3,080,960
│    │    └─Sequential: 3-26             785,280
│    │    └─Sequential: 3-27             147,456
│    │    └─Sequential: 3-28             258
│    └─Sequential: 2-18                  --
│    │    └─Sequential: 3-29             3,080,960
│    │    └─Sequential: 3-30             785,280
│    │    └─Sequential: 3-31             147,456
│    │    └─Sequential: 3-32             258
├─UpSample2x: 1-8                        --
==========================================================================
Total params: 33,625,412
Trainable params: 33,625,412
Non-trainable params: 0
==========================================================================
```

The appendix presents a comparison between HoverFast and HoverNet architectures. This information was produced using the Python Torchinfo Summary package. The first column outlines the model architecture, while the second delves into the number of parameters for each layer. It is noteworthy that HoverFast substantially smaller number of parameters (roughly 30 times fewer than HoverNet) which, along with the optimized post-processing and file handling, translates to lower memory footprint and faster processing time. This enables HoverFast to handle larger batches of data and allowing parallel post-processing computation, ultimately leading to a well-suited tool for resource limited environments.

References

Bankhead, P., Loughrey, M. B., Fernández, J. A., Dombrowski, Y., McArt, D. G., Dunne, P. D., McQuaid, S., Gray, R. T., Murray, L. J., Coleman, H. G., & others. (2017). QuPath: Open source software for digital pathology image analysis. *Scientific Reports*, *7*(1), 1–7. https://doi.org/10.1038/s41598-017-17204-5

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

Graham, S., Vu, Q. D., Raza, S. E. A., Azam, A., Tsang, Y. W., Kwak, J. T., & Rajpoot, N. (2019). Hover-net: Simultaneous segmentation and classification of nuclei in multi-tissue histology images. *Medical Image Analysis*, *58*, 101563. https://doi.org/10.1016/j.media.2019.101563

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv*

Preprint arXiv:1503.02531.

Hu, C., Li, X., Liu, D., Chen, X., Wang, J., & Liu, X. (2022). Teacher-student architecture for knowledge learning: A survey. *arXiv Preprint arXiv:2210.17332*.

Janowczyk, A., Zuo, R., Gilmore, H., Feldman, M., & Madabhushi, A. (2019). HistoQC: An open-source quality control tool for digital pathology slides. *JCO Clinical Cancer Informatics*, *3*, 1–7. https://doi.org/10.1200/CCI.18.00157

Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., & Lerer, A. (2017). *Automatic differentiation in PyTorch*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Schmidt, U., Weigert, M., Broaddus, C., & Myers, G. (2018). Cell detection with star-convex polygons. *Medical Image Computing and Computer Assisted Intervention - MICCAI 2018 - 21st International Conference, Granada, Spain, September 16-20, 2018, Proceedings, Part II*, 265–273. https://doi.org/10.1007/978-3-030-00934-2_30

Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: A generalist algorithm for cellular segmentation. *Nature Methods*, *18*(1), 100–106. https://doi.org/10.1038/s41592-020-01018-x

Su, R., Zhang, D., Liu, J., & Cheng, C. (2021). MSU-net: Multi-scale u-net for 2D medical image segmentation. *Frontiers in Genetics*, *12*, 639930. https://doi.org/10.3389/fgene.2021.639930

Weigert, M., Schmidt, U., Haase, R., Sugawara, K., & Myers, G. (2020). Star-convex polyhedra for 3D object detection and segmentation in microscopy. *The IEEE Winter Conference on Applications of Computer Vision (WACV)*. https://doi.org/10.1109/WACV45572.2020.9093435

Weinstein, J. N., Collisson, E. A., Mills, G. B., Shaw, K. R., Ozenberger, B. A., Ellrott, K., Shmulevich, I., Sander, C., & Stuart, J. M. (2013). The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, *45*(10), 1113–1120. https://doi.org/10.1038/ng.2764

DRAFT